Document Number:	P0055R1
Date:	2015-09-12
Project:	LEWG
Revises:	P0055R0
Reply to:	gorn@microsoft.com

P0055R1: On Interactions Between Coroutines and Networking Library

Introduction

Proposed Networking Library (N4478) uses the callback based asynchronous model described in N4045 which is shown to have lower overhead than the asynchronous I/O abstractions based on future.then ([4399]). The overhead of the Networking Library abstractions can be made even lower if it can take advantage of coroutines N4499. This paper suggests altering completion token transformation class templates described in N4478/[async.reqmts.async] to achieve near zero-overhead efficiency when used with coroutines. Changes in this revision clarify that the current CompletionToken model supports only one programming model efficiently, namely, continuation using callbacks whereas this proposal offers efficient mechanism supporting both callback and coroutine models.

Performance for callbacks and callbacks only

N4045: Library Foundation for Asynchronous Operations paper argues that std::future is a poor choice as a fundamental building block for asynchronous programming due to inherent performance limitations of std::future and suggests to use a callback model as a foundation mechanism. To support both users who desire to use callbacks and those who desire to use future-like objects, it offers a CompletionToken model adopted subsequently by Networking [P0112] and Executors [P0113] proposals.

We wholeheartedly agree with N4045 assessment of performance limitations of std::future. Unfortunately, the CompletionToken model proposed in N4045 and adopted by P0112 and P0113, does not offer any efficient mechanism for consumption of its APIs other than callbacks.

Without changes to traits similar to the ones proposed in this paper, one has to resort to utilizing `use_future` adapter described in P0112 that brings back inefficiencies related to future-based programming model. Indeed, a benchmark modeled after the one described in P0162, shows that overhead of the `use_future` mechanism results in nearly 50 times slower execution times than direct consumption of the async API via traits mechanism described in this paper.

```
// 7.9ns per iteration (as proposed)            // 390ns per iteration (using use_future)

std::future<void> loop() {                    |   std::future<void> loop() {
  for (int i = 0; i <= 100'000'000; ++i) {    |     for (int i = 0; i <= 100'000'000; ++i) {
    co_await async_xyz(0);                    |       co_await async_xyz(0, use_future);
  }                                           |     }
}                                             |   }
```

The tests were performed with use_future adapter mapping to light-weight `rexp::future` from https://github.com/chriskohlhoff/resumable-expressions. 26% of the time was spent in allocation/deallocation of promises and future shared objects, 20% was spent in synchronization primitives. Allocation overhead can be reduced with a custom allocator, however synchronization overhead is unavoidable in the current CompletionToken model. Proposed model allows to avoid both allocation and synchronization overhead, since it allows to defer launching of the operation until `.then` or `await_suspend` can provide completion callback to the API, allocation is avoided by allowing using a temporary on the coroutine frame that is stable in memory for the duration of the asynchronous operation.

Coroutines offer lower overhead than the callback model

Using the same benchmark as in P0162 and applied to slightly more sophisticated code (we included an index variable `i` to count number of iterations and a code to delete the state machine when desired number of iterations is reached), we reaffirmed the proposition of this paper that coroutines offer lower overhead than the callback model while allowing very compact and readable representation of an asynchronous state machine while maintaining or exceeding performance of the callback model.

```
                coroutine                          callback based equivalent

std::future<void> loop() {                  |  struct loop_state {
  for (int i = 0; i <= 100'000'000; ++i) {  |     int i = 0;
    co_await async_xyz(0);                  |     loop_state() {
  }                                         |       async_xyz(0, [this](OsResultType o) { OnComplete(o); });
```

```
}                                       |     }
                                        |     void OnComplete(OsResultType) {
                                        |       if (++i > 100'000'000) {
                                        |         delete this;
                                        |         return;
                                        |       }
                                        |       async_xyz(0, [this](OsResultType o) { OnComplete(o); });
                                        |     }
                                        |   };
                                        |   void loop() { new loop_state();}
```

Nanoseconds per iteration:

Test	/O2	/O2 /GL (whole program opt)
LLVM- Coro	**5.6ns**	6.1ns
VS 2015 Coro	6.2ns	**6.0ns**
Callback Default Alloc	6.8ns	6.7ns
Callback Custom Alloc	7.5ns	7.0ns

In this benchmark we used the callback model using a default thread caching allocator and a custom allocator that serves out memory from a preallocated fixed size arena (same as in P0162) and compared against two coroutine implementations, stock version from VS2015 Update 1 (VS 2015 Coro) and a coroutine implementation modeling coroutine optimization passes from an experimental llvm implementation (LLVM coro). As shown above, coroutines offer lower overhead that we expect to get better in the future as code generation and optimization strategies for coroutines improve with time.

Overview

Networking Library asynchronous functions uses class templates `completion_handler_type_t` and `async_result` to transform `CompletionToken` passed as a parameter to the interface functions starting with prefix async_ into a callable function object to be submitted to unspecified underlying implementation functions. This transformation allows to use the same set of functions whether using a callback model or relying on future based continuation mechanism. For the latter, an object of type `use_future_t` is provided in place of the callback parameter (for example: `async_xyz(buf, len, use_future)`).

```
template<class CompletionToken>
auto async_xyz(T1 t1, T2 t2, CompletionToken&& token)
{
  completion_handler_type_t<decay_t<CompletionToken>, void(R1 r1, R2 r2)>
    completion_handler(forward<CompletionToken>(token));

  async_result<decltype(completion_handler)> result(completion_handler);

  async_xyz_impl(t1, t2, completion_handler); // do the work

  return result.get();
}
```

We propose to use a single `completion_token_transform` function to perform transformation currently done via `completion_handler_type_t` and `async_result`. Not only this results in less boilerplate code for the user/library developer to write, but also enables zero-overhead mode when working with coroutines as described in the next section.

```
template<class CompletionToken>
auto async_xyz(T1 t1, T2 t2, CompletionToken&& token) noexcept(auto)
{
  return completion_token_transform<void(R1 r1, R2 r2)>(
       forward<CompletionToken>(token),
       [=](auto typeErasedHandler) { async_xyz_impl_raw(t1, t2, typeErasedHandler); });
}
```

Details

Let's explore how a high level asynchronous function `async_xyz` can be built on top of a low level `os_xyz` supplied by the platform. At first, we will write both callback and coroutine based solutions separately. Then, we will show how utilizing `completion_token_transform` as shown in the previous section allows the same API to handle efficiently both cases.

Let `ParamType` be the type representing all the input parameters to an asynchronous call, `ResultType` be the type of the result provided asynchronously and `OsContext*` is a pointer to a context structure `OsContext` that `os_xyz` requires to remain valid until the asynchronous operation is complete. The general shape

of the low level API is assumed to be as shown below.

```
using CallbackFnPtr = void(*)(OsResultType r, OsContext*); // os wants this signature
void os_associate_completion_callback(CallbackFnPtr cb); // usually per handle or per threadpool
void os_xyz(ParamType p, OsContext* o); // initiating routine (per operation)
```

To transform a call to `async_xyz(P, CompletionHandler)` into a call to `os_xyz`, we need to type erase the completion handler and pass it to the `os_xyz` as `OsContext*` parameter. In the completion callback, given an OsContext*, the callback will downcast it to the type containing the actual handler class and invoke it. In a simplified form it can look like:

```
template <typename CompletionHandler>
void async_xyz(ParamType p, CompletionHandler && cb) {
    auto o = make_unique<Handler<decay_t<CompletionHandler>>>(forward<CompletionHandler>(cb));
    os_xyz(p, o.get());
    o.release();
}

where Handler and HandlerBase defined as follows

struct HandlerBase : OsContext {
    CallbackFnPtr cb;
    explicit HandlerBase(CallbackFnPtr cb) : cb(cb) {}
    static void callback(ResultType r, OsContext* o) { // register this with OS
        static_cast<HandlerBase*>(o)->cb(r, o);
    }
};

template <typename CompletionHandler>
struct Handler : HandlerBase, CompletionHandler {
    template <typename CompletionHandlerFwd>
    explicit Handler(CompletionHandlerFwd&& h)
        : CompletionHandler(forward<CompletionHandlerFwd>(h))
        , HandlerBase(&Handler::callback)
    {}
    static void callback(ResultType r, OsContext* o) {
        auto me = static_cast<Handler*>(o);
        auto handler = move(*static_cast<CompletionHandler*>(me));
        delete me;  // deleting it prior to invoke improves allocator behavior
        handler(r); // as handle is likely to request a similar block which can be immediately reused
    }
};
```

While sophisticated implementations may utilize specialized allocation / deallocation functions to lessen the overhead of type erasure and memory allocations, the overhead cannot be eliminated completely in a callback model.

However, when asynchronous API is used in a coroutine, no type erasure or memory allocation needs to be performed at all. No only this results in less code and faster execution, it also eliminates the sole source of failure mode of async APIs allowing the library to mark async_xxx functions as noexcept.

Let's compare mapping `async_xyz` to an `os_xyz` when used in a coroutine. To be usable in an await expression (N4499/[expr.await]), `async_xyz(P, use_await_t)` function needs to return an object with member functions await_ready, await_suspend and await_resume defined as follows:

```
auto async_xyz(ParamType p, use_await_t = use_await_t{}) {
    struct Awaiter : AwaitableBase {
        ParamType p;
        explicit Awaiter(ParamType & p) : p(move(p)) {}

        bool await_ready() { return false; } // the operation has not started yet
        auto await_resume() { return move(this->result); } // unpack the result when done
        void await_suspend(coroutine_handle<> h) { // call the OS and setup completion
            this->resume = h;
            os_xyz(p, this);
        }
    };
    return Awaiter{ p };
}

where AwaitableBase defined as follows

struct AwaitableBase : HandlerBase {
    coroutine_handle<> resume;
```

```
    ResultType result;

    AwaitableBase() : HandlerBase(&AwaitableBase::Callback) {}

    static void Callback(ResultType r, OsContext* o) {
        auto me = static_cast<AwaitableBase*>(o);
        me->result = r;
        me->resume();
    }
};
```

The following example illustrates how a compiler transforms expression `await async_xyz(p)`.
Note the absence of memory allocations / deallocations and type erasure of any kind.

```
ResultType r = await async_xyz(p);

becomes

     async_xyz`Awaiter __tmp{p};
     $promise.resume_addr = &__resume_label;   // save the resumption point of the coroutine
     __tmp.resume = $RBP;                      // inlined await_suspend
     os_xyz(p,&OsContextBase::Invoke, &__tmp); // inlined await_suspend
     jmp Epilogue; // suspends the coroutine
__resume_label:    // will be resumed at this point once the operation is finished
     R r = move(__tmp.result); // inlined await_resume
```

Now with completion_token_transform

Given the public async function async_xyz defined as described in the Overview section (and repeated below for readers convenience)

```
template<class CompletionToken>
auto async_xyz(T1 t1, T2 t2, CompletionToken&& token) noexcept(auto)
{
  return completion_token_transform<void(R1 r1, R2 r2)>(
       forward<CompletionToken>(token),
       [=](auto typeErasedHandler) { async_xyz_impl_raw(t1, t2, typeErasedHandler); });
}
```

with the `completion_token_transform` defined as follows, we can achieve the same efficient implementation of asynchronous function when using callbacks:

```
template <typename Signature, typename CompletionHandler, typename Invoker>
void completion_token_transform(CompletionHandler && fn, Invoker invoker)
{
    auto p = make_unique<Handler<decay_t<CompletionHandler>>>(forward<CompletionHandler>(fn));
    invoker(p.get());
    p.release(); // if we reached this point, handler is owned by async activity and unique_ptr can relinquish the ownership
}
```

By defining overload for `use_await_t`, we can get efficient implementation of async_xyz when used in coroutines.

```
template <typename Signature, typename Invoker>
auto completion_token_transform(use_await_t, Invoker invoker)
{
    struct Awaiter : AwaiterBase, Invoker {
        bool await_ready() { return false; }
        ResultType await_resume() { return move(this->result); }
        void await_suspend(coroutine_handle<> h) {
            this->resume = h;
            static_cast<Invoker*>(this)->operator()(this);
        }
        Awaiter(Invoker& invoker) : Invoker(move(invoker)) {}
    };
    return Awaiter{ invoker };
}
```

And finally, for completeness, here is how `completion_token_transform` overload for `use_future_t` will look like:

```
template <typename Signature, typename Invoker>
auto completion_token_transform(use_future_t, Invoker invoker) {
```

```
    struct FutHandler {
        promise<ResultType> p;
        void operator()(ResultType r) { p.set_value(move(r)); }
    };
    auto p = make_unique<Handler<FutHandler>>(FutHandler{});
    auto f = p->p.get_future();
    invoker(p.get());
    p.release();
    return f;
}
```

Summary

Proposed changes improve efficiency of the networking library by altering the mechanism how high-level public API interprets CompletionToken when invoking unspecified internal implementation. If this direction has support, the author of this article will gladly help the author of Networking Library proposal to flesh out the relevant details and provide testing of proposed changes using coroutines available in MSVC compiler.

Future Work / Musing

There is an upcoming proposal (see [c++std-ext-17433]) to add [[nodiscard]] attribute/context-sensitive keyword to be applicable to classes and functions. If that attribute is applied to an awaiter class returned from the completion_token_transform, it will make it safe to add a default CompletionToken use_await_t to all async_xyz APIs.

```
template<class CompletionToken = use_await_t>
auto async_xyz(T1 t1, T2 t2, CompletionToken&& token  = use_await_t{}) noexcept(auto)
{
  return completion_token_transform<void(R1 r1, R2 r2)>(
       forward<CompletionToken>(token),
       [=](auto typeErasedHandler) { async_xyz_impl_raw(t1, t2, typeErasedHandler); });
}
```

If a user accidentally writes `async_xyz(t1,t2)` instead of `await async_xyz(t1,t2)`, the mistake will be caught at compile time due to `nodiscard` tag on the awaitable class.

Moreover, given that coroutines enable coding simplicity of synchronous functions combined with efficiency and scalability of asynchronous I/O, we may chose to use the nicest names, namely (send, receive, accept) to asynchronous functions and use `CompletionToken` form of the API to deal with all cases. A single API function `async_xyz` can be utilized for all flavors of operations. This shrinks required API surface by two thirds.

```
Instead of 3 forms of every API:

   void send(T1,T2);
   void send(T1,T2,error_code&);
   void async_send(T1,T2, CompletionToken);

We can use a single form

   auto send(T1,T2,CompletionToken);

To be used as follows:

   await send(t1,t2); // CompletionToken defaults to use_await_t as being the most efficient and convenient way of using the async A
   send(t1,t2,block); // synchronous version throwing an exception
   send(t1,t2,block[ec]); // synchronous version reporting an error by setting error code into ec
   send(t1,t2,[]{ completion }); // asynchronous call using callback model
   auto fut = send(t1,t2,use_future); // completion via future
```

Benefit of this approach extends beyond the networking library to other future standard or non-standard libraries modeling their APIs on the CompletionToken/completion_token_transform.

Acknowledgments

Great thanks to Christopher Kohlhoff whose N4045 provided the inspiration for this work.

References

N4045: Library Foundations for Asynchronous Operations, Revision 2

N4399: Technical Specification for C++ Extensions for Concurrency
N4478: Networking Library Proposal (Revision 5)
N4499: Draft Wording For Coroutines (Revision 2)
P0057: Wording for Coroutines
P0162: A response to "P0055R0: On Interactions Between Coroutines and Networking Library"